

NexAI

AI-Powered Personal & Developer Workspace — Web App (PWA)

Capstone PRD — Diploma Final Year (MSBTE) — Final Consolidated Version

0. What Changed From Earlier Versions

This replaces the Chrome-extension-only plan. NexAI is now a **full web app, installable as a PWA**, usable by anyone from any device/browser — not locked to Chrome. All previously designed features (bookmarks/library, AI chat, developer tools, focus tools, analytics, security, learning tools) are kept, just re-architected as web-app features instead of extension-only features. A few features that genuinely need in-browser access (tab session saving, site-time tracking, "ask AI" from a right-click on any page) move into a **small optional companion browser extension** that talks to the same backend — the PWA is the main product; the extension is a thin bridge, not a separate app.

1. Problem Statement

People collect information (links, documents, ideas, code snippets, AI-generated content) across many disconnected tools — bookmarks in one place, notes in another, AI chat history in a third, generated documents scattered across downloads. There's no single place to save, search, generate, and act on personal + professional knowledge using AI. Developers additionally need small daily utilities (JSON/regex tools, snippet storage) they currently reach for on ten different websites. Power users writing prompts for AI tools have nowhere to save and reuse them. And accessing an AI workspace quickly (without opening a browser tab and typing a URL) is still clunky compared to native OS assistants.

2. Product Vision

One AI workspace, several modes. The same account and data work across mode-based sections a user switches between:

- **General mode** — chat, document generation, personal library, productivity, learning, creative writing
- **Developer mode** — code tools, snippet manager, API tester, documentation helper
- **Student mode** — flashcards, quizzes, video/lecture summarizer, study notes

- **Power-user mode** — prompt vault, automation/reminders, analytics, security tools

Modes are just filtered views of the same sidebar — nothing is duplicated in the backend; a user can freely mix all of them.

3. Complete Feature List

A. Core AI Chat

- Text, image, file, and voice input; voice output (text-to-speech reply)
- Streaming responses
- Chats organized in a sidebar: **pin**, **rename**, **archive** (auto-deletes after 30 days), **delete**
- **Projects/Workspaces** — group related chats, notes, files, and documents under one project with its own custom instructions
- **Chat branching** — edit an earlier message and regenerate from that point without losing the original branch (both branches kept, switchable)
- Model choice: Gemini Flash (default, fast/cheap) vs Gemini Pro (complex/agentic tasks) — user-visible toggle per chat
- **Global instructions** (account-wide) + **per-project instructions** (override)
- Long-term memory: opt-in toggle, visible "what does it remember about me" review/edit screen, one-click "forget everything"
- Predefined prompt templates on the empty-chat welcome screen (customizable, pulls from the user's own Prompt Vault — see Section D)
- Search across **all** past chats (not just the open one), by keyword and by meaning (semantic)
- Export any chat as PDF or Markdown
- Shareable read-only link to a chat (optional, off by default)
- Web search: default-on toggle, shown per message when used, with source links
- Regenerate response / copy / "explain differently" / "make shorter" quick actions under each AI reply

B. Personal Knowledge Library

(this replaces "bookmark manager" — same functions, generalized beyond browser bookmarks)

- **Save to Library** — paste any link, or (with the companion extension) one-click save the current page, from any device
- Auto-summary + auto-tag + auto-categorize on save (Gemini Flash)
- **Semantic search** across saved links, uploaded files, notes, generated documents, and prompts — one search bar, one index (Pinecone)
- Folder + tag view (manual fallback alongside AI categorization)

- Smart deduplication (flags duplicate saved items)
- Link-rot + content-change monitor (scheduled check, flags dead or significantly changed pages)
- Reading queue with AI priority (High/Med/Low)
- "Related items" suggestions (k-NN similarity) shown when viewing any saved item
- Export/import the whole library as JSON or Markdown
- **AI Document Library** (new) — every document the AI generates for you (Section C) is automatically filed here, organized by category (Resume, Report, Letter, Notes, Other) with its own search and tags, separate from web-saved links but part of the same unified search

C. AI Document Generation (*new — detailed*)

Flow:

1. User asks in chat or from a dedicated "Create Document" screen: *"write me a resume", "generate a report on X", "draft a cover letter for Y"*.
2. AI drafts structured content as sections (title, headings, body, optional bullet lists/tables) — shown in a live **document preview pane**, not just chat text.
3. User can edit any section inline before exporting.
4. **Images are never added automatically.** Each section shows an **"+ Add image"** button. Only when clicked does the user choose:
 - Upload their own image, **or**
 - Search free stock photos (Unsplash/Pexels free API), **or**
 - Generate an AI image for that spot (flagged as the one feature with a real per-image cost — see Section 10; used sparingly, always by explicit click)
5. One-click **Export** → PDF or DOCX, downloaded immediately.
6. The finished document is auto-saved into the **AI Document Library** (Section B) under an appropriate category, with the option to regenerate/edit later.
 - Supports common templates: Resume, Cover Letter, Report/Essay, Meeting Notes, Study Notes, Letter — plus a free-form "blank AI document" option.
 - Versioning: regenerating a document keeps the previous version accessible (simple version history, not full diffing).

D. Prompt Vault (*new — detailed*)

- Save any prompt — from a past chat message ("save this prompt") or written fresh in the vault directly
- Organize with **tags/categories** (e.g. "coding," "writing," "study")
- **Variables/placeholders** in saved prompts — e.g. *Summarize this article about {{topic}} in {{tone}} tone* — filled in via a small form when reused, so one saved prompt becomes a reusable template
- **One-click "Use this prompt"** — inserts it (with variables filled) directly into a new or existing chat

- Favorite/pin frequently used prompts; recently used list
- Search within the vault
- Import/export vault as JSON (portability, backup)
- A small **starter pack** of built-in example prompts on first use (local, no external content dependency)

E. Files & Document Analysis

- Upload PDF/Word/Excel/images → ask questions about them, get summaries
- Compare two documents — highlight what changed/differs
- Meeting transcript summarizer → key points + action items
- OCR support for scanned/image-based PDFs (Gemini's multimodal input handles this directly, no separate OCR service needed)

F. Productivity & Life Assistant

- Task/to-do list with AI-suggested priority
- Reminders — "remind me about this bookmark/document/task in N days" (Web Push notification)
- Email/message drafting assistant (multiple tone variants, like the message-drafting pattern you'd expect from a real AI assistant)
- Resume/cover-letter builder (uses Section C's document generation engine)
- Weekly AI digest — summarizes what was saved/generated/learned that week, shown as an in-app card and optional push notification

G. Learning & Study

- Flashcard generator (from any note, document, or saved link) with spaced-repetition review scheduling
- Quiz generator from your own uploaded material
- Video/lecture summarizer with auto-generated timestamps (paste a YouTube link)
- "Explain this documentation/code" — paste a snippet, get a breakdown (what it does, parameters, logic, real-world example) — matches a clear teaching style, reusable for any confusing text

H. Creative & Writing

- Long-form writing assistant (blog posts, essays, reports) with adjustable tone/style
- Quick inline actions on any pasted text: rewrite / shorten / expand / change tone / fix grammar
- Idea/brainstorm mode — structured brainstorming with follow-up questions instead of one flat answer

I. Developer Utilities

- **Code Snippet Manager** — save/tag/search reusable code snippets with syntax highlighting
- **Quick Dev Tools panel**: JSON formatter/validator, regex tester, Base64 encode/decode, JWT decoder, UUID/hash generator, color picker + palette generator, cron expression explainer, Markdown live previewer, HTTP status code lookup, npm package quick-lookup (free npm registry API)
- **API request tester** — a mini-Postman inside the app (send GET/POST, view response)
- GitHub repo quick-save → auto-fetch README summary
- Stack Overflow question quick-save → auto-summarized accepted answer

J. Focus & Sessions

- **Workspace Sessions** — save a named group of open items (tabs, if the companion extension is installed; otherwise a manually curated group of saved links) and restore them together later
- Focus Timer (Pomodoro-style) with an in-app "focus mode" that hides non-essential UI
- Site/app blocklist during a focus session — **requires the companion browser extension** (a pure web app cannot block other sites)
- Quick-capture inbox — one keystroke to dump a thought, triage later

K. Analytics & Insights

- Usage of the NexAI app itself: documents created, chats had, items saved, flashcards reviewed — always available, no extension needed
- **Site-level time tracking** (time-per-domain, focus score, weekly trend) — **requires the companion browser extension**, since a web app cannot see time spent on other websites
- AI-based category labeling of tracked domains (work/dev/social/entertainment)
- Optional goals ("limit distracting sites to 30 min/day") with progress bar

L. Security & Privacy Tools

- **Secrets vault** — encrypted (Web Crypto API) storage for API keys/tokens, encrypted client-side before ever touching the network; even the backend only ever stores ciphertext
- Password strength / breach checker via the free HaveIBeenPwned k-anonymity API
- Privacy dashboard: trackers/cookies blocked per page — **requires the companion browser extension**
- **Export all my data / Delete all my data** — full account data portability and right-to-delete, one click each

M. Search & Discovery

- One unified, global search bar across chats, library items, documents, snippets, notes, and prompts
- Semantic ("find by meaning") and keyword modes
- Instant search-as-you-type with live preview

N. Voice & Accessibility

- Voice mode: speak to the assistant, hear it reply (Web Speech API — free, browser-native)
- Text-to-speech read-aloud for any saved article or generated document
- Adjustable font size, high-contrast mode, reduced-motion mode

O. Quick Access / PWA Launcher

- Installable as a **PWA** (Add to Home Screen / Install App) for an app-like window, offline shell, and fast reopen
- **Global quick-access shortcut** (the "Windows Copilot / Win+C" experience you asked for): a small always-on-top search/ask bar you can summon instantly.
 - **Important technical note:** a pure web app (even installed as a PWA) cannot register a true *system-wide* keyboard shortcut on its own — browsers don't expose that to web pages for security reasons. The realistic way to get this exact behavior is through the **companion browser extension**, which *can* register a global shortcut (Chrome's **commands** API, e.g. **Ctrl+Shift+Space**) that instantly opens a small popup — a mini version of the app (ask a quick question, search the library, jump to a saved prompt) — backed by the same account/backend as the full PWA.
 - Without the extension installed, the PWA itself still opens fast from the taskbar/dock/home-screen icon (a light substitute, not a true global hotkey).
- Command palette *inside* the app (**Ctrl/Cmd+K**) to jump to any feature instantly — this one works fully in the web app, no extension needed.

P. Settings & Personalization

- Theme (dark default) + accent color, font size, language
- Keyboard shortcuts reference/customization
- Model choice (Flash/Pro) default, memory on/off, web-search-default on/off
- Quiet/focus hours (no notifications during set hours)
- Notification preferences (which events trigger a push notification)
- Data & privacy: export data, delete data, review what's remembered

Q. Onboarding

- First-run interactive tutorial covering the sidebar sections
- Contextual tips ("try asking me to write a document," "save this chat's prompt to your vault")
- Empty-state screens with a clear call-to-action instead of blank panels

R. Optional Companion Browser Extension (Phase-later, thin bridge only)

Only exists to do what a pure web app structurally cannot:

1. One-click "save current page" to the Library
2. Right-click → "Ask AI about this" (selected text sent to the app)
3. Workspace Session tab-saving (reads actual open tabs)
4. Site-level time tracking + focus-score data feed
5. Global quick-access hotkey (Section O)
6. Site blocklist during focus sessions

Everything else in this PRD works fully in the web app with **no extension required** — this keeps the core product usable by anyone on any browser/device, with the extension as a genuine bonus for power users.

4. Non-Goals / Deliberately Cut

- Multi-provider model marketplace (Gemini/Claude/GPT switcher), billing/subscription UI, a public plugin marketplace, third-party connectors beyond the free public APIs already listed (GitHub, npm, HavelBeenPwned, Unsplash/Pexels), team/multi-user collaboration.
- Reason: these serve a general-purpose consumer product's business needs, not this project's problem-solving goal — depth on save/generate/organize/ask beats breadth of settings screens, and keeps you inside free-tier infra with a smaller attack surface for bugs before your viva.

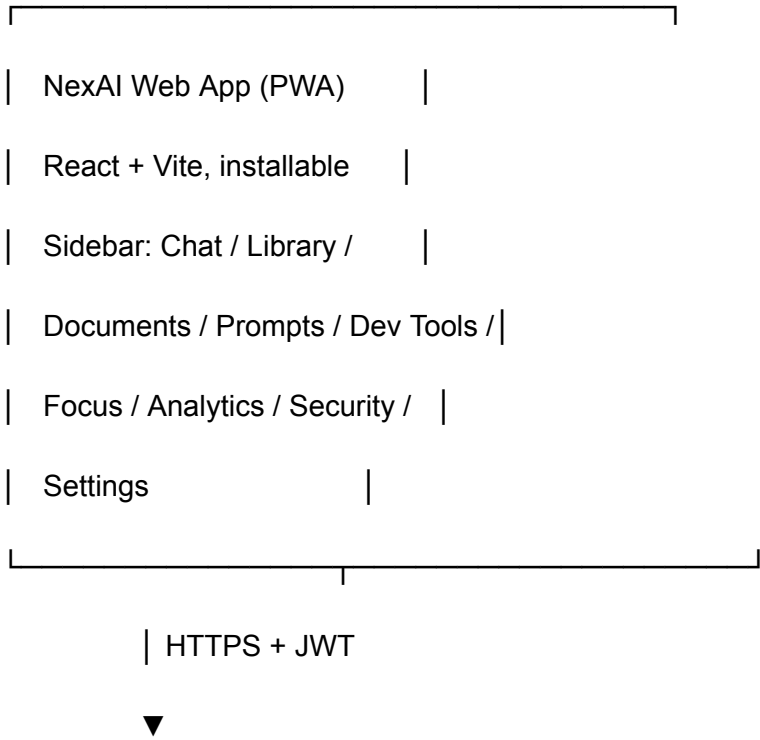
5. Tech Stack

Layer	Choice	Notes / Free-tier fit
Frontend (PWA)	React + Vite, SCSS modules, Zustand, Workbox (service	\$0

Layer	Choice	Notes / Free-tier fit
	worker for installability/offline shell)	
Rich text/document editor	Tiptap	\$0, open-source
Backend	Node.js + Express	Render free web service
AI orchestration	LangChain / LangGraph	\$0 (library)
LLM	Gemini 1.5 Flash (default: chat, tagging, summarization, document drafting) + Gemini 1.5 Pro (complex/agentic tasks)	Your existing Pro-tier key
Embeddings	Gemini embedding model	Same key
Vector DB	Pinecone (Starter/free) — 2 GB storage, 5 indexes, 2M writes/mo, 1M reads/mo	Plenty for a single-user demo dataset
Database	MongoDB Atlas M0 — 512 MB, free forever	Stores chats, library items, documents metadata, prompts, snippets
Auth	Google OAuth + JWT	\$0, works identically for web app and extension
PDF/DOCX generation	pdf-lib (PDF) + docx npm package (Word) — pure JS, no headless browser needed	Chosen over Puppeteer specifically because Puppeteer's headless Chrome is too memory-heavy for Render's free tier
Images in documents	User upload (free) / Unsplash or Pexels free API (free) / Gemini image generation (paid per image — the one place free-tier can break; keep opt-in and rare, exactly as you specified)	

Layer	Choice	Notes / Free-tier fit
Push notifications	Web Push API (VAPID, free)	For reminders/digest, no third-party notification service needed
Hosting — web app	Vercel free (Hobby)	\$0
Hosting — backend	Render free web service (spins down after 15 min idle, ~30–60s cold start)	Mention as a known trade-off in your viva
Scheduled jobs	<code>node-cron</code> inside Express	Link checks, reminders, digest generation
Companion extension (optional)	Manifest V3, same React components reused where possible	\$0, published unlisted or side-loaded for demo purposes

6. Architecture Overview





| Companion Browser Extension | — same backend, same JWT session —>
(backend above)

| (tab session save, site time |

| tracking, global hotkey, "Ask AI |

| about this page", site blocklist) |

|_____|

Key design decisions carried forward:

- All AI calls go through the backend — the Gemini key never touches the client.
- The agent layer uses defined tools (`saveToLibrary`, `searchLibrary`, `generateDocument`, `checkLink`, `webSearch`) rather than one giant prompt — this is the concrete answer to "isn't this just an API wrapper?"
- Any agent action that changes data follows **suggest** → **review** → **confirm** — nothing is silently written.
- Image generation inside document creation is the **only** feature gated behind an explicit click, by design, both for cost control and because you specifically asked for on-demand-only images.

7. Data Model

// users

{

 _id, email, googleId,

 settings: {

 theme, accentColor, fontSize, language,

 defaultModel: "flash" | "pro",

 memoryEnabled: Boolean,

 webSearchDefaultOn: Boolean,

```
    globalInstructions: String,

    quietHours: [{ day, start, end }],

    notificationPrefs: { reminders: Boolean, digest: Boolean }

},

createdAt

}

// projects

{ _id, userId, name, instructions: String, createdAt }

// libraryItems (generalized "bookmark")

{

    _id, userId, projectId,

    type: "link" | "file" | "note",

    url, title, faviconUrl, summary, tags: [String], folder,

    status: "active" | "broken", priority: "high" | "medium" | "low",

    vectorId: String,      // Pinecone reference

    lastCheckedAt, createdAt

}

// documents (AI-generated documents — Section C)

{

    _id, userId, projectId,

    title, category: "resume" | "report" | "letter" | "notes" | "other",

    sections: [{ heading, body, imageUrl }], // imageUrl null unless user added one
```

```
    fileFormat: "pdf" | "docx",

    versionHistory: [{ versionAt, sections }],

    createdAt, updatedAt
  }

// prompts (Prompt Vault — Section D)

{
  _id, userId,

  title, template: String,      // e.g. "Summarize {{topic}} in {{tone}} tone"

  variables: [String],

  tags: [String], pinned: Boolean, lastUsedAt,

  createdAt
}

// chats / messages (unchanged structure)

{
  _id, userId, projectId, title, pinned, archived, archivedAt, createdAt, updatedAt
}

{
  _id, chatId, role: "user" | "assistant" | "tool",

  content, toolCalls: [{ name, input, output }], createdAt
}

// snippets
```

```

{ _id, userId, title, language, code, tags: [String], createdAt }

// sessions (Workspace Sessions — needs extension for real tab data)

{ _id, userId, name, items: [String], createdAt } // items = URLs

// flashcards

{ _id, userId, sourceId, sourceType, question, answer, nextReviewAt, easeFactor, createdAt }

// usageStats (app-level always; site-level only with extension)

{ _id, userId, date, domainMinutes: [{ domain, minutes, category }], appActivity: {}, focusScore }

// reminders

{ _id, userId, targetId, targetType, remindAt, sent: Boolean }

// agentTasks (Phase 2 research agent)

{ _id, userId, type, status: "pending" | "awaiting_review" | "done", input, proposedResult,
createdAt }

```

The **secrets vault** intentionally has no readable-plaintext schema anywhere — only ciphertext is ever stored, encrypted client-side with a key derived from the user's own passphrase (never sent to the server).

8. Key Data Flows

A. Save to Library

1. User pastes a link or uploads a file (or the extension sends the current page).
2. Backend fetches/extracts content → Gemini Flash generates summary + tags.
3. Embedding generated → upserted to Pinecone → **libraryItems** doc saved with **vectorId**.
4. Suggested tags/summary shown for one-tap confirm/edit — never applied silently.

B. Chat with RAG

1. User asks a question in chat.
2. Query embedded → Pinecone top-k search across library + documents + notes.

3. Retrieved context + chat history passed to the LangGraph agent.
4. Agent answers directly or calls a tool; response streamed back; message pair saved.

C. AI Document Generation

1. User requests a document (chat or dedicated screen).
2. Agent drafts structured sections (JSON: heading/body per section).
3. Rendered in the preview pane; user edits inline.
4. User optionally clicks "+ Add image" per section → chooses upload / free-stock search / AI-generate.
5. On export click → `pdf-lib/docx` renders the final file → downloaded → auto-saved to `documents` + indexed into the Library's unified search.

D. Prompt Vault use

1. User opens the vault, picks a saved prompt.
2. If it has `{{variables}}`, a small form collects values.
3. Filled prompt inserted into a new/existing chat as the first message.

E. Quick-access launcher (with extension)

1. User presses the global hotkey anywhere in the OS.
2. Extension opens a small popup, authenticated via the same JWT/session.
3. User types a question or search term → same backend endpoints as the main app respond → shown in the popup, with a "open full app" link for anything needing more space.

F. Broken-link check (scheduled)

1. Daily cron pulls library items not checked in 7+ days.
2. HEAD request per URL; failures marked `status: "broken"`; user notified in-app/push.

9. Folder Structure

nexai/

├── web-app/ # Main PWA client

| ├── src/

- | | | — pages/ # Chat, Library, Documents, Prompts, DevTools,
- | | | # Focus, Analytics, Security, Settings
- | | | — components/
- | | | — store/ # Zustand stores
- | | | — lib/ # api client, auth, pdf/docx export helpers
- | | | — pwa/ # service worker, manifest.json, install prompt
- | | | — styles/ # SCSS, dark theme tokens
- | | — public/
- | | — vite.config.js
- |
- | — extension-companion/ # Optional, thin — not required for core product
- | | — src/
- | | | — background/ # global hotkey (chrome.commands), tab/session sync
- | | | — content-scripts/ # "Ask AI about this", site-time tracking
- | | | — popup/ # quick-access mini UI
- | | — manifest.json
- |
- | — server/
- | | — src/
- | | | — config/
- | | | — models/ # Mongoose schemas
- | | | — routes/

```

| | | └─ controllers/
| | | └─ services/          # gemini.service.js, pinecone.service.js,
| | |                       # documentExport.service.js (pdf-lib/docx)
| | | └─ agents/           # LangGraph graph + tool definitions
| | | └─ jobs/             # linkChecker.cron.js, digest.cron.js, reminders.cron.js
| | | └─ middleware/
| | └─ app.js
| └─ package.json
|
└─ docs/
    | └─ PRD.md             # this file
    | └─ architecture-diagram.png
    | └─ viva-prep.md
    |
    └─ README.md

```

10. Free-Tier Reality Check

Service	Free limit	Practical impact
MongoDB Atlas M0	512 MB, free forever	Enough for thousands of items across all collections
Pinecone Starter	2 GB, 5 indexes, 2M writes/mo, 1M reads/mo	Comfortably covers a single-user demo

Service	Free limit	Practical impact
Render free web service	750 instance-hrs/mo, ~30–60s cold start after idle	Mention proactively in your viva
Vercel Hobby	Generous static hosting	\$0
GitHub public API	60 req/hr unauthenticated, 5,000/hr with a token	Fine for repo-summary lookups
HavelBeenPwned API	Free k-anonymity lookup	No key needed
Unsplash/Pexels API	Free tier, rate-limited	Covers on-demand stock images
Web Speech / Web Crypto / Web Push	Built into the browser	\$0
pdf-lib / docx npm packages	Open-source	\$0, avoids Puppeteer's memory cost on Render free tier
Gemini API	Your existing Pro-tier key	Text generation, embeddings — already covered
Gemini image generation	Not free	The <i>only</i> real cost item — kept opt-in/on-demand exactly as specified, so cost stays near-zero in normal use

Everything except occasional AI-generated images runs at \$0 beyond the Gemini key you already have.

11. Phased Build Roadmap

Phase 0 — Core MVP: Chat (A) + Library save/search/summarize/tag (B) + basic Settings (P). This alone is a complete, defensible capstone.

Phase 1 — Differentiators: AI Document Generation (C) + Prompt Vault (D) + unified Search (M) + Command palette. These are what make it feel like a "real AI app," not a bookmark tool.

Phase 2 — Depth: Files & Document Analysis (E), Developer Utilities (I), Learning & Study (G), Productivity (F), Onboarding (Q).

Phase 3 — Power-user layer: Analytics (K, app-level parts), Security tools (L), Creative writing (H), Voice/Accessibility (N).

Phase 4 — Extension bridge (only if time remains): Companion extension for tab sessions, site-time tracking, global hotkey, site blocklist, "Ask AI about this page."

Viva rule of thumb: have Phase 0–1 fully working, Phase 2 mostly working, and at least a working demo of the global-hotkey extension in Phase 4 if possible — it's the single most impressive live demo moment.

12. Answering the Examiner Questions

- **"Isn't this just an API wrapper?"** — The value is the retrieval pipeline (embeddings + Pinecone across chats/library/documents/prompts), the document-generation pipeline (structured sections → on-demand images → PDF/DOCX export), the Prompt Vault's variable system, and the suggest→confirm agent pattern — none of that is "just calling an API."
- **"Why a web app and not just use ChatGPT/Gemini directly?"** — Because it's a single, searchable personal knowledge base tied to *your* saved links, documents, and prompts, with generation and organization fused together, installable as an app, and (optionally) reachable via a global hotkey like a native OS assistant.
- **"Is this a real-world problem?"** — Yes: fragmented personal knowledge across bookmarks/notes/chat history/downloaded documents is a common, documented pain point, not an invented one.

Be ready to explain in your own words: why Pinecone over plain keyword search, why images are gated behind a click (cost + user intent), and what you personally designed vs what a library/API does for you.